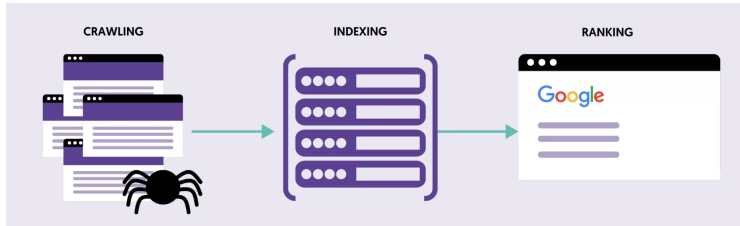


Introduction to Information Retrieval

Procheta Sen



Anatomy of a Search Engine



Steps of Indexing

- ▶ Format Detection: Documents can be in different format (e.g. XML, flat text).
- ▶ Text Extraction
- ▶ Language Detection
- ▶ Tokenisation
- ▶ Index Construction

Tokenisation

- ▶ If we do not tokenise properly, then we cannot search for those terms!
- ▶ Token refers to a single unit of text that we can search for.
- ▶ Tokens might not necessarily be words in English.
- ▶ Sentence: 'the burger i ate was an awesome burger'
Tokens = the, burger, i, ate, was, an, awesome, burger
- ▶ Repeating tokens are counted separately. Note the two burger tokens in the previous example.
- ▶ Sentence: 'U.S.A. isn't easy-to-parse!'
Tokens: 'U', 'S', 'A', 'isn', 't', 'easy', 'to', 'parse'

Is Tokenisation Simple?

- ▶ Just split at spaces to get tokens `s.split()`
- ▶ What about the following?
Dr. D. T. Bollegala
Should we consider Dr, . , D, ., T, ., Bollegala or Dr., D., T., Bollegala?
- ▶ Japanese and Chinese languages do not use spaces at all!
- ▶ Modern day we have different variations of tokenizer with respect to different pretrained models.

Index Construction

- ▶ Index is a large table between tokens and unique document ids.
- ▶ Similar to the indexes you find at the end of a textbook.
Support Vector Machines p. 13, p. 56, p. 124.
- ▶ Search engines create an inverted index from.

D_1 = I went to school

D_2 = The school was closed

D_3 = Tomorrow is a holiday

I	D_1
went	D_1
school	D_1 D_2
tomorrow	D_3
holiday	D_3
closed	D_2
to	D_1

The list of document IDs for a particular token is called a *posting list*

Retrieval Models

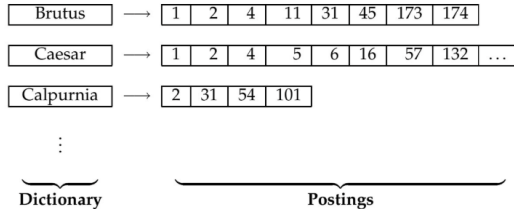
Boolean Retrieval Model

- ▶ Assign integer IDs to documents so that we can sort the posting lists. (e.g. $D_1 \implies 1, D_2 \implies 2$)
- ▶ Queries are in Boolean format (e.g. School AND Holiday).
- ▶ We only need to store one entry for a word in a document. (multiple entries can be ignored).
- ▶ Queries can use operators like AND, OR, NOT
- ▶ Start with the shorter posting list.
- ▶ Find the document ids in the shorter list in the longer list.

Boolean Retrieval Model Example

Example

Query = *Brutus* AND *Calpurnia*
Results = 2, 31



Start with the posting list for Calpurnia (shorter list), and check for 2, 31. No point checking any further than 173. We will not find 54 and 101 in Brutus.

Issues With Boolean Retrieval Model

- ▶ The notion of similarity does not translate directly into an assessment of 'is the document a good document to give to the user or not?'
- ▶ The most similar document can be highly relevant or completely nonrelevant.
Let's say the query is "Apple laptop",
 D_1 : 'MacBook Air offers up to 18 hours of battery life.'
 D_2 : 'Apple trees require plenty of sunlight for fruit production.'
Here D_1 and D_2 has same similarity but, D_2 is not at all similar.
- ▶ Probability theory is arguably a cleaner formalization of what we really want an IR system to do: give relevant documents to the user.

Probabilistic Ranking Principle (PRP)

- ▶ Assume binary relevance: $R_{d,q}$ is a random dichotomous variable representing if document d is relevant w.r.t query q . otherwise.
- ▶ Often we write just R for $R_{d,q}$.
- ▶ Probabilistic ranking orders documents decreasingly by their estimated probability of relevance w.r.t. query ($R_{d,q}$) .
- ▶ Assume that the relevance of each document is independent of the relevance of other documents.
- ▶ PRP in brief: If the retrieved documents (w.r.t a query) are ranked decreasingly on their probability of relevance, then the effectiveness of the system will be the best that is obtainable.

Binary Independence Model: a PRP Based Retrieval Model

- ▶ Document d is represented as $x = (x_1, x_2 \dots x_t)$ where x_i is a token present in a document.
- ▶ Equation for Relevance: $P(R = 1|x, q) = (P(x|R = 1, q)P(R = 1|q))/P(x|q)$
Equation for Non-Relevance: $P(R = 0|x, q) = (P(x|R, q)P(R = 0|q))/P(x|q)$
- ▶ $P(R = 0|q)$ is the prior probability of retrieving a non-relevant document for a query. Same is $P(R = 1|q)$.
- ▶ $P(x|R = 1, q)$ is the probability that x is retrieved as a relevant document for a query.
- ▶ $P(R = 1|x, q) + P(R = 0|x, q) = 1$

Binary Independence Model (Continued)

- ▶ Since we are interested only in the ranking of documents, we can rank documents by their odds of relevance.
- ▶ $O(R|X, q) = \frac{P(R=1|x, q)}{P(R=0|x, q)} = \frac{P(R=1|q)}{P(R=0|q)} \frac{P(x|R=1, q)}{P(x|R=0, q)}$
- ▶ The left term in the above expression is a constant term for a query. So we are only interested in the right hand term. We use Naive Bayes conditional independence assumption that the presence or absence of a word in a document is independent of the presence or absence of any other word (given the query):
- ▶ $\frac{P(x|R=1, q)}{P(x|R=0, q)} = \prod_{t=1}^M \frac{P(x_t|R=1, q)}{P(x_t|R=0, q)}$
- ▶ Retrieval Status Value (RSV) is computed.
$$RSV_d = \log \prod_{t: x_t = q_t = 1} \frac{p_t}{u_t} \prod_{t: x_t = 0, q_t = 1} \frac{(1-p_t)}{(1-u_t)}$$
- ▶ p_t =the term appears in a relevant document, u_t =the term appears in a non-relevant document.
- ▶ BIM still do not consider term frequency, which is a problem

Okapi BM25

- ▶ Okapi BM25 is a probabilistic model that incorporates term frequency (i.e., it's nonbinary) and length normalization.
- ▶ BIM was originally designed for short catalog records of fairly consistent length, and it works reasonably in these contexts. For modern full-text search collections, a model should pay attention to term frequency and document length.
- ▶ BestMatch25 (a.k.a BM25 or Okapi) is sensitive to these quantities.
- ▶ BM25 is one of the most widely used and robust retrieval models.

Okapi BM25 (Continued)

- ▶ The formula for BM25 is as follows.

$$RSV^{BM25} = \sum_{i \in q} \log\left(\frac{N}{df_i}\right) \cdot \frac{(k_1 + 1) \cdot tf_i}{k_1((1 - b) + b \cdot \frac{dl}{avgdl}) + tf_i} \quad (1)$$

- ▶ N = total number of documents in the corpus.
- ▶ df_i = document frequency of term i , number of documents in which term i appears.
- ▶ tf_i = term frequency of term i , number of times term i appears in a document
- ▶ dl is document length, $avgdl$ is average document length.
- ▶ k_1 controls term frequency scaling, $k_1 = 0$ is binary model
- ▶ b controls document length normalization, $b = 0$ is no length normalization;
- ▶ $b = 1$ is relative frequency (fully scaled by document length)

Language Models in IR

- ▶ language model based retrieval in the late 1990s and early 2000s (mainly by Ponte & Croft - A Language Modeling Approach to Information Retrieval).
- ▶ Here document d is represented as a generative model that generates the query q .
- ▶ Ranking is done by how likely the query is under the document's word distribution $P(q|d)$.